Optimization & Uncertainty in Learning Report
CS7641: Machine Learning
Summer 2026

1 Assignment Weight

The assignment is worth 15% of the total points.

Read everything below carefully as this assignment has changed term-over-term.

2 Objective

Most machine learning projects do not fail because a model is incapable of learning, they fail because we pull the wrong training levers, in the wrong order, and then mistake coincidence for causation. This assignment asks you to stop treating your neural network as a black box.

Using the **PyTorch neural network you finalized in the SL Report as a fixed Covertype backbone**, you will study how optimization and regularization choices affect training dynamics, stability, runtime, and multiclass generalization. The goal is not simply to improve Accuracy. The goal is to explain how specific training choices affect validation loss, Macro-F1, balanced accuracy, class-level behavior, and compute cost under controlled budgets.

In this assignment, uncertainty is treated operationally through seed sensitivity, trajectory variance, stability bands, and minority-class performance variability under fixed compute budgets. You will use these tools to distinguish a stable training improvement from a lucky run.

You will interrogate three forces that shape modern training:

1. **Randomized Optimization (RO)** applied to the final 1–3 layers under a strict parameter budget to examine robustness, tail behavior, and local search dynamics.
2. **Adam-family optimizer ablations** on the full network to separate the contributions of momentum, Nesterov acceleration, adaptive scaling, bias correction, and decoupled weight decay.
3. **Regularization** using standard Adam to quantify how explicit regularization choices change validation loss, test metrics, class-level behavior, and stability.

Every comparison must be conducted under fixed, reported budgets so that differences are causal rather than accidental. You will report gradient evaluations, function evaluations, and wall-clock time where appropriate. You will conclude by composing a practical decision rule from the elements that proved most effective on Covertype and by explicitly accepting or rejecting your hypotheses with quantitative evidence.

You will:

- Reuse the Covertype sample, preprocessing, split strategy, and PyTorch neural-network backbone from the SL Report.
- Apply randomized optimization to the final 1–3 layers of the SL PyTorch network under a strict parameter cap.
- Dissect Adam-family optimization through controlled ablations on the full network.
- Run a targeted regularization study using standard Adam with the best Part 2 hyperparameters.

- Compare optimizer changes, regularization changes, and RO fine-tuning under fixed compute budgets.
- Integrate the evidence into a coherent conclusion about what training choices mattered most and why.

3 Connection to the SL Report

This assignment builds directly on your Supervised Learning Report. You must reuse the same Covertype working dataset, target, train/validation/test split, preprocessing strategy, and PyTorch neural-network backbone from the SL Report.

The purpose of this requirement is experimental control. OL should test optimization and regularization choices, not introduce a new dataset, new feature pipeline, new split, or new architecture search. If you change anything material from SL, such as cleaning rules, feature handling, label mapping, sample construction, split strategy, preprocessing, or architecture, you must disclose and justify the change. You must also explain how the change affects comparison to the SL baseline.

Your **SL PyTorch SGD-only neural network** is the baseline reference point for this assignment. Comparisons to the SL baseline should use the same sample, split, preprocessing, architecture, and test set whenever possible. If any part differs, the comparison to SL must be marked as approximate and the change must be justified.

4 Evaluation Overview

The full rubric will be used by the TAs during grading, but the table below summarizes the main areas you should expect to be evaluated on. The report is not graded only on whether a method achieves the highest score. It is graded on whether the experiments are controlled, reproducible, budget-aware, visually readable, and interpreted carefully.

Category	**Points**	**What this means**
Compliance	16	Correct Covertype sample and `Cover_Type` target, reuse of the SL PyTorch backbone, Enterprise Overleaf and GT Enterprise GitHub access, reproducible run instructions, experiment details in REPRO, readable figures and tables, peer-reviewed sources, and AI Use Statement.
Study design	10	Leakage-safe splits, held-out test discipline, training-only validation, clear compute accounting, consistent seed protocol, early hypothesis, and hypothesis resolution using quantitative evidence.
Randomized Optimization	18	RHC, SA, and GA on the final 1–3 layers under the parameter cap, deterministic validation objective, function-evaluation accounting, budget-matched progress curves, and interpretation of whether RO was worth the compute.
Adam-family ablations	20	Required optimizer variants, fair optimizer protocols, loss-versus-compute analysis, threshold analysis, sensitivity heatmaps or summaries, seed stability, and interpretation of optimizer ingredients.
Regularization study	16	Standard Adam fixed from Part 2, required regularizers tested and disclosed, budget-matched baseline/single/combination comparisons, sweep evidence, and interpretation relative to optimizer effects.
Discussion	20	Cross-part synthesis, compute-performance trade-offs, optimization versus generalization versus stability, class-level behavior, limitations, and a practical decision rule for Covertype.
Total	**100**	
Extra Credit	+5	Integrated best combination using the best Adam settings, best regularization combination, and best RO fine-tuning method under constrained additional search.

The most common weak report will run many experiments but fail to explain what changed, what was held fixed, whether the change was stable, whether the figures can be read clearly, and whether the extra compute was justified. A strong report may find only modest improvements, but it will explain those results carefully using validation loss, test metrics, class-level behavior, stability across seeds, and compute cost.

5 Required Dataset for Summer 2026

You will reuse the **Forest Cover Type / Covertype** dataset from your SL Report. If this dataset is not used, or if `Cover_Type` is not treated as a seven-class multiclass classification target, you will receive a zero for the assignment.

Forest Cover Type / Covertype

- **Task type:** Multiclass classification
- **Target:** `Cover_Type`, classes 1 through 7
- **Working dataset:** Approximately 20,000 instances sampled from the full dataset, matching the SL Report sample unless a change is explicitly justified
- **Required metrics:** Validation loss, Accuracy, Macro-F1, balanced accuracy, confusion matrix, and per-class precision/recall/F1 discussion

6 Procedure

You will complete one optimization-focused study on the Covertype multiclass classification task. You must keep the PyTorch SL Report backbone architecture fixed unless a specific part of the assignment permits a documented change.

You will:

1. confirm that the Covertype sample, preprocessing, split, and PyTorch backbone match the SL Report,
2. establish the SL PyTorch SGD-only baseline for comparison,
3. run randomized optimization on the final 1–3 layers under a strict parameter budget,
4. run Adam-family optimizer ablations on the full network,
5. run regularization studies using standard Adam,
6. compare all interventions under fixed compute budgets, and
7. write a coherent analysis explaining which training choices mattered most and why.

Hypothesis-driven analysis requirement

Your EDA and dataset assumptions should be consistent with what you reported in the SL Report so that comparisons remain meaningful. You may summarize EDA more briefly in this report, but you must preserve the same core facts and assumptions used to motivate your SL baseline.

After confirming the dataset and SL baseline, you must propose at least **one clear hypothesis** about expected optimization behavior before running full experiments. This hypothesis should be grounded in theory from the lectures and readings, such as optimization stability, conditioning, momentum effects, adaptive step sizes, bias correction, regularization as capacity control, and robustness under stochastic training.

A strong hypothesis does not merely predict which method will win. It predicts a training behavior or trade-off. For example, you might hypothesize that Adam-family methods will reach a validation-loss threshold faster than SGD but that regularization will affect Macro-F1 or minority-class recall more than optimizer choice. You might hypothesize that RO fine-tuning will improve validation loss only marginally relative to its compute cost. Your experiments and discussion should be structured around testing these claims.

You must revisit this hypothesis later in the report. The hypothesis should be accepted, rejected, or qualified using quantitative evidence from at least two types of evidence, such as validation loss, test Macro-F1, balanced accuracy, class-level behavior, seed stability, gradient evaluations, function evaluations, or wall-clock time.

General rules

- You may program in Python and are allowed to use standard libraries, **as long as they were not written specifically to solve this assignment**.
- Code repositories, notebooks, or AutoML templates created for CS7641 assignments ("plug-and-play" solutions) are **not allowed**.
- TAs must be able to recreate your experiments on a standard Linux machine if necessary.
- The analysis you provide in the report is paramount and the focus of the assignment.

PyTorch-only backbone requirement

For this assignment, you must use **only the PyTorch neural network** from your SL Report. Do not use the scikit-learn MLP from the SL Report for the OL experiments.

This requirement exists to ensure that optimizer ablations, regularization studies, and RO fine-tuning are controlled, comparable, and interpretable. The PyTorch implementation gives you direct control over training loops, optimizers, loss functions, regularization modules, seed behavior, and epoch-level diagnostics.

7 Parts and Requirements

Part 1: Randomized Optimization on Final Layers

Goal: Evaluate whether search-based methods can improve or stabilize the final-layer behavior of your Covertype PyTorch network under a strict parameter budget.

Freeze all but the final 1–3 trainable layers so that the total number of trainable parameters for randomized optimization is $\leq\sim$ 50,000. Report the exact trainable parameter count. If the final 1–3 layers exceed this budget, reduce the number of trainable layers or use a smaller final-layer configuration that preserves the SL backbone as much as possible.

The objective is validation loss. If evaluating a regularized condition, include the regularization terms consistently. Count every objective call as a *function evaluation*. If evaluating the objective on the full validation set is computationally prohibitive, you may use a fixed, stratified validation subset for objective calls, but you must disclose this choice and still perform final evaluation on the full held-out test set.

Required algorithms:

- Randomized Hill Climbing (RHC)
- Simulated Annealing (SA)
- Genetic Algorithm (GA)

Required disclosures:

- **RHC:** restart policy, step-size schedule, perturbation scale, and stopping rule.
- **SA:** initial temperature, decay factor, cooling schedule, step-size cooling, and stopping rule.
- **GA:** population size, selection method, crossover operator, mutation rate, elitism policy, and stopping rule.
- **Objective calls:** how validation loss was computed, whether a fixed validation subset was used, and how function evaluations were counted.

Required analysis:

- best-so-far validation objective versus function evaluations,
- validation loss versus wall-clock time,
- final test Accuracy, Macro-F1, balanced accuracy, and class-level behavior,

- compute trade-off relative to the SL PyTorch baseline and gradient-based optimizers,
- discussion of whether RO improved performance, improved stability, or mainly consumed extra compute.

Part 2: Adam-Family Optimizer Ablations

Goal: Dissect which optimizer ingredients affect training speed, stability, and generalization on the full Covertype PyTorch backbone.

Train the same full network with the following optimizers:

- **SGD with no momentum**
- **SGD with momentum**
- **Nesterov momentum**
- **Adam**
- **Adam without bias correction**
- **Adam with** $\beta_1 = 0$ as an RMSProp-like condition
- **AdamW** with decoupled weight decay

The Adam without bias correction condition may require a custom implementation or a carefully adapted optimizer. If you implement a custom optimizer, you must document the update equations or provide clear code references in the reproducibility materials.

Required analyses:

- validation loss versus wall-clock time,
- validation loss versus gradient evaluations or optimizer updates,
- time and steps to a fixed validation-loss threshold ℓ,
- sensitivity heatmaps over coarse (α, β_1) and (α, β_2) grids,
- stability across at least 3 seeds unless computationally justified,
- generalization gap at budget, using train versus validation loss or metric,
- final test Accuracy, Macro-F1, balanced accuracy, and class-level behavior.

Choose ℓ using only training/validation evidence. A reasonable choice is a validation-loss threshold reached by at least one optimizer during preliminary runs, or a fixed relative improvement over the SL baseline. Do not choose ℓ using test performance.

Heatmaps should be coarse and interpretable rather than exhaustive. A 3×3 or 4×4 grid is generally more useful than an expensive fine grid that prevents seed analysis or interpretation. The main report should include compact heatmap summaries or the most informative heatmap. Extended heatmaps may be placed in the REPRO PDF.

Part 3: Regularization Study

Goal: Measure how regularization affects Covertype generalization under a fixed optimizer and fixed compute budget.

Use **standard Adam** with the best Part 2 hyperparameters. Do not change Adam settings in this part. Use the PyTorch SL Report backbone. Only add, remove, or alter regularization mechanisms, and document those changes precisely.

Required techniques:

1. **L2 weight decay:** select and justify a small search over weight-decay values.

2. **Early stopping:** define patience, monitored quantity, and restore-best behavior if used.
3. **Dropout:** document layer placement and dropout rates.
4. **Noise regularization:** evaluate label smoothing and modest input noise or augmentation applied only during training.

Required comparisons:

- **Adam baseline:** standard Adam with no added regularization.
- **Best single regularizer:** the strongest individual technique under the fixed budget.
- **Best regularization combination:** the strongest combination under the same fixed budget.

Required analysis:

- validation loss and test metrics under the same compute budget,
- whether regularization improved Macro-F1, balanced accuracy, minority-class recall, validation loss, or only stability,
- whether regularization reduced overfitting as measured by train-validation gaps,
- whether any regularizer harmed specific classes,
- how much regularization moved the test metric relative to the Part 2 Adam baseline.

Detailed regularization sweeps may be placed in the REPRO PDF. The main report must include compact summaries and interpretation of the most important regularization findings.

Part 4: Integrated Best Combination (Extra Credit)

Optional (up to 5 points). If you choose to complete Part 4, you must integrate all three components and report how the final recipe compares to the rest of your results.

What to do:

- **Optimizer:** Use standard Adam with the best hyperparameters from Part 2 and the same batch size.
- **Regularization:** Apply the best regularization combination from Part 3, with exact values and layer placements.
- **RO fine-tuning:** Apply your best-performing RO algorithm from Part 1 to the same final 1–3 layers under the same parameter cap used in Part 1.

Constraints:

- Do not introduce new optimizers or fresh wide hyperparameter sweeps.
- Limit RO fine-tuning to $\leq 10\%$ of the Part 1 RO function-evaluation budget.
- Limit interaction exploration to ≤ 5 small trials, such as minor dropout-rate adjustments.
- Keep seeds, hardware class, data splits, and batch size consistent with earlier parts.

What to report explicitly: whether RO fine-tuning on top of the best Adam and regularization combination improves test metrics beyond the best result from Parts 1–3; the compute trade-off in test metric versus total evaluations and time; and any stability effects observed across seeds. End with a concise hypothesis resolution paragraph that accepts or rejects your hypotheses with quantitative evidence.

8 What is Required as You Go Through Your Analysis

Data, baseline, and target

Declare the Covertype target and use the same sample, preprocessing, and split strategy as the SL Report unless a change is explicitly justified. Report enough EDA confirmation to show that the OL study is comparable to SL.

Your baseline is the SL PyTorch SGD-only neural network. Explicitly compare outcomes back to this baseline and state whether each intervention improved, failed to improve, or improved only under certain compute or stability conditions.

Methodology and splits

Use one held-out test set only at the end. Tune only on training and validation data. Set and log all random seeds, exact splits, deterministic flags where applicable, and hardware class. Keep the PyTorch backbone fixed unless a specific part permits a regularization change.

Preprocessing must remain leakage-safe. Scaling, feature transformations, and any other learned preprocessing operations must be fit only on training data or training folds, then applied to validation and test data.

Compute accounting and fairness

Report gradient evaluations for SGD/Adam-family methods, function evaluations for RO methods, and wall-clock time. Keep batch size, data splits, hardware class, and evaluation protocol consistent across comparisons unless a change is necessary and justified.

For gradient-based methods, define what counts as an update or gradient evaluation. For example, one mini-batch backward pass may count as one gradient evaluation/update. If you use full-batch training, gradient accumulation, or unusual batching, disclose how counts are computed.

For RO methods, count every validation objective call as a function evaluation. If the objective is evaluated on a fixed validation subset, disclose the subset size, sampling method, and seed.

Clearly mark over-budget runs and exclude them from head-to-head comparison tables.

Required figures and tables

At minimum, the main report should include compact versions of the following:

- **Baseline comparison:** SL PyTorch baseline versus best OL interventions.
- **Loss vs compute:** validation loss versus wall-clock and evaluations.
- **RO progress:** best-so-far objective versus function evaluations.
- **Optimizer trajectories:** validation loss or metric over compute for Adam-family variants.
- **Sensitivity summary:** compact heatmap or summary of (α, β_1) and (α, β_2) behavior.
- **Stability summary:** median/IQR, variance bands, or compact seed summary.
- **Regularization summary:** Adam baseline, best single regularizer, and best combination.
- **Final metric table:** method, best validation loss, test Accuracy, test Macro-F1, test balanced accuracy, wall-clock, gradient evaluations, function evaluations, and relevant notes.
- **Class-level summary:** confusion matrix or per-class precision/recall/F1 for the most important comparisons.

Extended heatmaps, full seed tables, complete regularization sweeps, and additional diagnostic plots may be placed in the REPRO PDF. The main report must still include compact summaries and interpretation of the most important findings. Do not move all substantive analysis to the REPRO PDF.

Discussion and conclusion

Make claims only with budget-matched evidence. Explain divergence, plateaus, instability, or class-level failures. Discuss whether improvements are meaningful in terms of test metrics, validation loss, class-level behavior, stability, and compute cost.

Your Discussion should synthesize across Part 1, Part 2, and Part 3. It should not simply summarize each part separately. A strong Discussion should explain which intervention primarily improved optimization speed, which intervention primarily affected generalization, which intervention changed stability across seeds, and which intervention, if any, helped class-level behavior. It should also explain when validation-loss improvements did or did not transfer to test Macro-F1, balanced accuracy, or minority-class recall.

Your Discussion must distinguish **optimization**, **generalization**, **stability**, and **efficiency**. These are not the same claim. A method that reaches a lower validation loss faster is not necessarily the method with the best test Macro-F1. A method that improves seed stability may be valuable even if its mean test score changes only slightly. A method that improves Accuracy may still be weak if minority-class recall declines.

Your Discussion must include a concrete limitations analysis. Good limitations are specific to your study design and evidence. Examples include the approximately 20,000-instance sample, class imbalance, instability in minority-class estimates, limited RO function-evaluation budget, final-layer-only RO scope, coarse Adam heatmaps, limited seed count, validation-subset use for RO, hardware constraints, or the fact that the conclusion depends on the SL backbone architecture. A limitation should not be a generic statement such as "more tuning could be done" unless it is tied to specific evidence from your results.

End with a practical decision rule for Covertype. This decision rule should explain when to prioritize optimizer changes, when to prioritize regularization, and whether RO fine-tuning is worth the compute. It should also state the limits of transfer. For example, a recipe that works well for Covertype may not transfer directly to another dataset with different class imbalance, feature scaling, or sample size.

9 External Sources and Literature (required)

Include outside sources beyond course materials, with at least **two peer-reviewed references**. These should be used to support your methodological choices or interpretation, not merely listed in the references section.

Primary sources are strongly preferred. In this context, a primary source is usually an original peer-reviewed paper that introduces, evaluates, or studies a method, optimizer, regularization technique, metric, or evaluation issue directly. For example, a paper on Adam, AdamW, dropout, label smoothing, randomized optimization, imbalanced classification metrics, or leakage-safe evaluation is more appropriate than a broad summary article.

Avoid relying on textbooks for this requirement. Textbooks can be useful for learning background material, but they are usually too general for supporting specific experimental choices in this report. Your outside sources should help justify decisions such as why Macro-F1 is appropriate for imbalanced multiclass classification, how Adam's bias correction or adaptive scaling affects optimization, why decoupled weight decay differs from coupled L2 regularization, or how dropout and early stopping affect generalization.

Peer review matters. Avoid citing arXiv papers, preprint servers, blog posts, documentation pages, tutorials, lecture notes, Wikipedia, Medium posts, or vendor documentation as one of your required peer-reviewed references. These may be useful for implementation help or background reading, but they should not replace the required scholarly sources.

Use **IEEE citation style**. A strong report uses sources in the body of the paper to support claims. Do not only place two papers in the bibliography. Cite them where they help justify a metric, evaluation protocol, optimizer interpretation, regularization choice, leakage control, or interpretation of results.

10 Report Requirements

Your report is a formal technical paper limited to **8 pages total**, including figures and references. Anything past 8 pages will not be read.

Your main report must use the **IEEE conference template**. The main report should use the official IEEE conference paper format, such as:

```
\documentclass[conference]{IEEEtran}
```

Your report must be written in LaTeX on Overleaf using your **Georgia Tech / Enterprise Overleaf account**. When submitting your report, you are required to include a **READ-ONLY** link to the Overleaf project. Do not share the project directly via email.

Main report versus REPRO

The main report must stand alone as a formal technical paper. It should include the central experimental design, compact figures/tables, key results, and interpretation.

The REPRO PDF may contain extended implementation details, full heatmaps, full seed tables, detailed regularization sweeps, additional plots, and full run logs. These materials support reproducibility and grading, but they do not replace interpretation in the main report.

A good main report does not show everything. It shows the most important evidence and explains what it means.

Interpret, do not summarize

Every figure and table must include a takeaway tied to optimizer behavior, regularization behavior, RO dynamics, class-level behavior, or compute trade-offs. Avoid repeating the legend. Explain what the result *means* and why it happened.

For example, do not merely state that Adam reached a lower validation loss than SGD. Explain whether Adam reached that loss faster, whether the improvement was stable across seeds, whether it transferred to Macro-F1 or balanced accuracy, and whether the improvement justified the compute.

Formal writing requirement

Your *report* must read like a technical paper, not a checklist. Bullets are allowed when they improve clarity, such as listing hyperparameters, summarizing preprocessing steps, or describing an experimental protocol. However, your **analysis and conclusions must be written in paragraphs** with coherent reasoning.

Excessive bullet-only writing that replaces interpretation, especially in Results, Discussion, or Conclusion sections, will cause the report to be treated as a draft and scored accordingly. Multiple instances of four or more consecutive bullets in analysis-heavy sections should be treated as excessive unless they are clearly part of a compact protocol, hyperparameter list, or reproducibility note.

11 Acceptable Libraries

You may use any standard open-source libraries that support reproducible experimentation and were not written specifically to solve this assignment. The list below includes common examples.

Recommended stack

- **PyTorch:** required for the neural networks and optimizer ablation work in this assignment.
- **scikit-learn:** useful for splits, metrics, preprocessing utilities, and evaluation summaries.
- **imbalanced-learn:** useful for imbalance utilities if justified.
- **pyperch:** helpful for parameter search and includes PyTorch-compatible optimizers modeled after ABAGAIL.
- **Data and plotting:** `pandas`, `NumPy`, and `matplotlib`; `plotly`, `altair`, or `seaborn` are also acceptable.

Randomized Optimization libraries

The following libraries may be used for randomized optimization, but library use does not remove the obligation to disclose operators, hyperparameters, and objective-call accounting:

- **mlrose-hiive** — RHC, SA, GA tooling.
- **DEAP** — evolutionary algorithm tooling.
- **Nevergrad** — derivative-free optimization toolbox; use only for RHC, SA, or GA-like operators that you disclose.
- **pyperch** — useful for PyTorch-compatible randomized optimization workflows.

If you use any library tooling, you must still disclose operator choices and count every objective call as a function evaluation.

Custom optimizer implementations

Some Adam ablations, especially Adam without bias correction, may require a custom implementation. This is allowed, but you must document the update rule, cite relevant sources, and provide code that can be inspected and reproduced. If your custom optimizer differs from the intended ablation, disclose the difference and interpret results accordingly.

12 Submission Details

All scored assignments are due by the time and date indicated on Canvas (Eastern Time, ET). Canvas displays times in your local time zone if your profile is set correctly; grading deadlines are enforced in ET.

All assignments are due at 11:59:00 PM ET on the final Sunday of the unit. Submissions received by **7:59:00 AM ET** the next morning are accepted **without penalty**.

Late penalty

After the grace window, the score is reduced **20 points per calendar day** starting at 7:59:00AM ET day-over-day. Treat **11:59 PM ET** as your real deadline, allow time for upload and verification.

What to submit

You will submit **two PDFs**:

1. **OL_Report_{GTusername}.pdf** — your formal IEEE-style report.
2. **REPRO_OL_{GTusername}.pdf** — your reproducibility sheet.

The reproducibility sheet must include:

1. A **READ-ONLY** link to your Overleaf project.
2. A **GitHub commit hash** from your final code submission.
3. Exact run instructions to reproduce results on a standard Linux machine.
4. Environment setup, package versions, commands, data paths, and random seeds.
5. Confirmation that the Covertype sample, preprocessing, splits, and PyTorch backbone match the SL Report, or a clear explanation of any changes.
6. Full compute accounting details, including gradient evaluations, function evaluations, and wall-clock measurements.
7. Extended heatmaps, full seed tables, detailed regularization sweeps, and additional diagnostics that do not fit in the main report.

- Include the READ-ONLY Overleaf link in the report or Canvas submission comment. **Do not send email invitations.**

- Use the **GT Enterprise GitHub** for course-related code.
- Personal GitHub repositories are not acceptable for this assignment.
- Provide sufficient instructions to retrieve code and data. Canvas paths and file names are sufficient when applicable.

13 Feedback Requests

When your assignment is scored, you will receive feedback explaining errors and successes. If you are confused by feedback, start a private thread on Ed.

14 Plagiarism and Proper Citation

The easiest way to fail this class is to plagiarize. **Using the analysis, code, or graphs of others in this class is considered plagiarism**. We care about your analysis: it must be original and grounded in your own experiments.

If you copy any amount of text from other students, websites, or any other source without proper attribution, that is plagiarism. Citing is required but does not permit copying large blocks of text. All citations must use IEEE style for this assignment.

LLMs (disclosure required)

We treat AI-based assistance the same way we treat collaboration with people. You may discuss ideas and seek help from classmates, colleagues, and AI tools, but all submitted work must be your own. The goal of reports is synthesis of analysis, not merely getting an algorithm to run.

Every submission must include an AI Use Statement. List the tools used and what they assisted with and confirm that you reviewed and understood all assisted content.

Allowed with disclosure: brainstorming, outlining, grammar and clarity edits, code generation, code refactoring, and debugging.

Not allowed: submitting AI-written analysis, conclusions, or figures as your own; fabricating results or citations; paraphrasing AI or prior work to evade checks.

Example Statement at the very end of the report before References: "AI Use Statement. I used ChatGPT and Visual Studio Code Copilot to brainstorm and outline sections of the report, generate and refactor small code snippets, debug an indexing issue, and edit grammar and clarity throughout. I reviewed, verified, and understand all assisted content."

15 Version Control

- v1.0 – 05/19/2026 – TJL refactored OL Report for Summer 2026 Covertype assignment.

Assignment description written by Theodore LaGrow.